

UNIT 1 — Software Engineering & Agile Methodology

PART A — 1 Mark Answers

1. Define Software

Software is a collection of programs, procedures, and related documentation that instructs a computer to perform specific tasks.

2. What is Software Engineering?

Software Engineering is a systematic and disciplined approach used for designing, developing, testing, and maintaining software systems.

3. Who is a Software Engineer?

A software engineer is a professional who designs, develops, tests, and maintains software applications using engineering principles.

4. List one characteristic of good software

Reliability. (Correctness, Reliability, Efficiency, Maintainability, User-friendliness, Security, Scalability)

5. What is Software Crisis?

Software crisis refers to the problems faced in software development such as high cost, delays, poor quality, and difficulty in maintenance.

6. What does SDLC stand for?

Software Development Life Cycle.

7. What is a Software Process?

A software process is a structured set of activities used to develop and maintain software systems.

8. What is the main idea of the Waterfall Model?

Development progresses sequentially through phases such as requirement, design, coding, testing, and maintenance.

9. What is the key focus of the Spiral Model?

Risk analysis and iterative development.

10. Define Agile Methodology

Agile is an iterative and incremental software development approach that focuses on flexibility, customer collaboration, and continuous delivery.

11. What is Extreme Programming (XP)?

Extreme Programming is an Agile method that improves software quality through practices like pair programming and test-driven development.

12. When was the Agile Manifesto introduced?

2001.

13. What is a Sprint in Scrum?

A sprint is a short development cycle (usually 2–4 weeks) in which a specific set of tasks is completed.

14. What is a self-organizing team?

A team that manages its own work and decides how to complete tasks without external control.

PART B — 2 Marks

1. Differentiate between Software and Hardware

Software	Hardware
Program instructions	Physical components
Intangible	Tangible
Easily modified	Difficult to modify

2. Two objectives of Software Engineering

- To produce high-quality software
 - To complete software within budget and time
-

3. Two characteristics of good software

- Reliability
 - Maintainability
-

4. Two causes of Software Crisis

- Increasing complexity of software
 - Poor project management
-

5. Phases of SDLC

1. Requirement Analysis
 2. Design
 3. Implementation
 4. Testing
 5. Deployment
 6. Maintenance
-

6. Software Process vs SDLC

Software Process

SDLC

Software Process

SDLC

Overall development activities Specific lifecycle phases

Defines workflow

Defines stages

7. Incremental Model

Software is developed in small parts called increments. Each increment delivers partial functionality.

8. Prototyping Model Use

Used when requirements are unclear or not well understood.

9. Two features of Agile

- Iterative development
 - Continuous customer feedback
-

10. Role of Product Owner

Represents the customer and manages the product backlog.

PART C — 3 Marks

1. Importance of Software Engineering

Software engineering is important because it:

- Improves software quality
 - Reduces development cost
 - Helps manage complex systems
 - Ensures reliable and maintainable software
-

2. Software Crisis

Software crisis refers to problems faced during software development.

Problems include:

- Delays in project completion
- High development cost

- Poor quality software
- Difficult maintenance

Software engineering was introduced to solve these issues.

3. Phases of SDLC

1. Requirement Analysis
2. System Design
3. Implementation
4. Testing
5. Deployment
6. Maintenance

These phases ensure systematic software development.

4. Waterfall Model

The Waterfall Model is a sequential development process where each phase must be completed before the next phase begins.

Advantages

- Simple to understand
- Well structured

Disadvantages

- No flexibility
 - Difficult to change requirements
-

5. Spiral Model

The Spiral Model combines iterative development with risk analysis.

Each loop includes:

- Planning
- Risk Analysis
- Engineering
- Evaluation

It is suitable for large projects.

6. Agile Methodology

Agile is a flexible development approach where software is built in small iterations.

Benefits:

- Faster delivery
 - Better customer satisfaction
 - Easy requirement changes
-

7. Extreme Programming Practices

- Pair Programming
 - Test Driven Development
 - Continuous Integration
 - Small Releases
 - Refactoring
-

PART D — 4 Marks

1. Compare Process Models

Model	Feature
Waterfall	Sequential development
Incremental	Development in parts
Prototyping	Prototype built first
Spiral	Risk driven model
Agile	Iterative and flexible

2. SDLC with Diagram

Requirement → Design → Coding → Testing → Deployment → Maintenance

SDLC ensures systematic development of software.

3. Agile Manifesto Values

1. Individuals over processes
 2. Working software over documentation
 3. Customer collaboration over contract negotiation
 4. Responding to change over following a plan
-

4. Scrum Framework

Scrum consists of:

Roles

- Product Owner
- Scrum Master
- Development Team

Artifacts

- Product Backlog
- Sprint Backlog
- Increment

Events

- Sprint Planning
 - Daily Scrum
 - Sprint Review
 - Sprint Retrospective
-

5. Extreme Programming

XP focuses on fast development and high quality.

Advantages

- Fast development
- Continuous testing

Disadvantages

- Not suitable for large teams.

UNIT-2

Requirement Engineering & Design Concepts

PART A — 1 Mark Answers

1. What is Requirement Engineering?

Requirement Engineering is the systematic process of gathering, analyzing, documenting, validating, and managing the requirements of a software system.

2. Define Requirement Modeling.

Requirement modeling is the process of representing system requirements using diagrams and models to help understand system functionality clearly.

3. What is an SRS document?

SRS (Software Requirement Specification) is a document that describes the functional and non-functional requirements of a software system.

4. What are functional requirements?

Functional requirements describe the specific functions or services that the system must perform.

5. What are non-functional requirements?

Non-functional requirements define system quality attributes such as performance, security, reliability, and usability.

6. What is a stakeholder?

A stakeholder is any person or organization that has an interest in the software system.

7. What is requirement elicitation?

Requirement elicitation is the process of collecting requirements from stakeholders.

8. What is validation in Requirement Engineering?

Validation is the process of checking whether the requirements are correct, complete, and feasible.

9. Define Software Design.

Software design is the process of converting system requirements into a blueprint for building software.

10. What is Software Architecture?

Software architecture defines the overall structure of the system and relationships between its components.

11. What is cohesion?

Cohesion refers to how closely related the functions inside a module are.

12. What is coupling?

Coupling refers to the level of dependency between different modules.

13. What is a Use Case Diagram?

A Use Case Diagram shows the interaction between users (actors) and the system.

14. What is a Class Diagram?

A Class Diagram represents classes, attributes, methods, and relationships between classes.

15. What is a Sequence Diagram?

A Sequence Diagram shows how objects interact in a particular sequence of time.

16. What is an Activity Diagram?

An Activity Diagram represents the workflow of activities in a system.

PART B — 2 Marks Answers

1. Seven Tasks of Requirement Engineering

The requirement engineering process includes the following tasks:

1. **Inception** – Identifying stakeholders and project scope.
 2. **Requirement Elicitation** – Collecting requirements from stakeholders.
 3. **Elaboration** – Analyzing and refining requirements.
 4. **Negotiation** – Resolving conflicts between stakeholders.
 5. **Specification** – Documenting requirements in SRS.
 6. **Validation** – Checking correctness and feasibility.
 7. **Requirement Management** – Managing requirement changes.
-

2. Functional vs Non-Functional Requirements

Functional Requirements	Non-Functional Requirements
Define system behavior	Define system quality
Describe system functions	Describe system constraints
Example: Login system	Example: System response time

3. Components of an SRS Document

Main components include:

1. **Introduction** – Purpose and scope of the system.
 2. **Overall Description** – System overview and assumptions.
 3. **Functional Requirements** – System functions.
 4. **Non-Functional Requirements** – Performance, security etc.
 5. **External Interface Requirements** – Interaction with users or systems.
-

4. Requirement Validation

Requirement validation ensures that the requirements are:

- Correct
- Complete
- Consistent
- Feasible

Validation methods include:

- Reviews
- Prototyping
- Test case generation

5. Characteristics of a Good SRS

A good SRS should be:

- Correct
- Complete
- Consistent
- Unambiguous
- Verifiable

6. Design Concepts in Software Engineering

Design concepts are principles used to structure software systems effectively.

Examples:

- Abstraction
- Modularity
- Cohesion
- Coupling

7. Abstraction and Modularity

Abstraction

Abstraction hides internal details and shows only essential features.

Example: ATM machine hides internal banking processes.

Modularity

Modularity divides a system into smaller modules.

Advantages:

- Easier maintenance
- Better debugging

8. Cohesion vs Coupling

Cohesion

Relation inside module

High cohesion is good

Coupling

Relation between modules

Low coupling is good

9. Architectural Design

Architectural design defines the overall system structure and interactions between components.

Examples:

- Layered architecture
 - Client-server architecture
 - MVC architecture
-

10. Purpose of UML Diagrams

UML diagrams help to:

- Visualize system structure
 - Represent system behaviour
 - Improve communication between developers
-

PART C — 3 Marks Answers

1. Seven Tasks of Requirement Engineering

Requirement engineering is the process of identifying and managing system requirements.

The seven tasks include:

1. Inception

Identifying stakeholders and defining system objectives.

2. Requirement Elicitation

Gathering requirements using techniques like interviews, surveys, and observation.

3. Elaboration

Analyzing and refining gathered requirements.

4. Negotiation

Resolving conflicts between stakeholders.

5. Specification

Documenting requirements in the SRS document.

6. Validation

Ensuring requirements are correct and feasible.

7. Requirement Management

Handling requirement changes throughout the project.

2. Gathering Functional and Non-Functional Requirements

Functional requirements define system functions such as login, registration, and payment.

Non-functional requirements define system quality attributes such as:

- Performance
- Security
- Reliability
- Usability

Requirements are collected using:

- Interviews
 - Questionnaires
 - Workshops
 - Observation
-

3. Requirement Modeling

Requirement modeling is the representation of system requirements using diagrams.

Types of requirement models include:

1. **Scenario-based modeling** – Use Case diagrams.
2. **Class-based modeling** – Class diagrams.
3. **Behavioral modeling** – Sequence diagrams.
4. **Flow-oriented modeling** – Data Flow Diagrams.

Benefits:

- Improves understanding
 - Helps detect missing requirements
 - Enhances communication.
-

4. Structure of SRS Document

The SRS document contains:

1. Introduction

Purpose and scope of the system.

2. Overall Description

General system characteristics.

3. Functional Requirements

Detailed system functions.

4. Non-Functional Requirements

Performance, reliability, and security requirements.

5. External Interfaces

User interface, hardware interface, and software interface.

SRS serves as an agreement between the client and developers.

5. Design Principles

Important design principles include:

DRY (Don't Repeat Yourself)

Avoid duplication of code.

KISS (Keep It Simple)

Keep the design simple.

High Cohesion

Each module should perform a single task.

Low Coupling

Modules should be independent of each other.

PART D — 4 Marks

1. Requirement Engineering in Detail

Requirement engineering is the process of identifying, documenting, validating, and managing system requirements.

Steps include:

1. Inception
2. Requirement Elicitation

3. Elaboration
4. Negotiation
5. Specification
6. Validation
7. Requirement Management

Requirement engineering ensures that the developed software meets user expectations and reduces development errors.

2. Software Requirement Specification (SRS)

SRS is a formal document describing system requirements.

Structure:

1. Introduction
2. Overall Description
3. Functional Requirements
4. Non-Functional Requirements
5. Interface Requirements

Importance:

- Acts as a contract between client and developer
 - Helps in project planning
 - Reduces misunderstandings
-

3. Software Design Concepts

Important design concepts include:

Abstraction – Hiding unnecessary details.

Modularity – Dividing system into modules.

Cohesion – Degree of relation within module.

Coupling – Dependency between modules.

A good design should have **high cohesion and low coupling**.

4. Architectural Design

Architectural design defines system structure.

Types include:

Layered Architecture

Presentation Layer

Business Layer

Data Layer

Client-Server Architecture

Client sends request → Server processes → Response returned.

MVC Architecture

Model – Data

View – User interface

Controller – Business logic

5. UML Diagrams

Important UML diagrams include:

Use Case Diagram

Shows interaction between user and system.

Class Diagram

Represents system structure.

Sequence Diagram

Shows message exchange over time.

Activity Diagram

Represents system workflow.

UNIT-3

Software Quality Assurance & Maintenance

PART A — 1 Mark Answers

1. Define Quality in Software Engineering

Quality refers to the degree to which software meets specified requirements and user expectations.

2. What is SQA?

Software Quality Assurance is a systematic process that ensures software meets defined quality standards.

3. What is a Software Review?

A software review is an evaluation of software documents or code to detect defects.

4. What is Formal Technical Review?

FTR is a structured review process conducted by experts to identify errors.

5. Define Software Reliability

Software reliability is the probability that software performs without failure for a specified period.

6. What is ISO 9000?

ISO 9000 is an international standard for quality management systems.

7. What does CMM stand for?

Capability Maturity Model.

8. What is Six Sigma?

Six Sigma is a quality improvement methodology focused on reducing defects.

9. What is SQA Plan?

An SQA Plan is a document that describes quality assurance activities in a project.

10. Define Software Maintenance

Software maintenance is the process of modifying software after delivery.

11. One type of software maintenance

Corrective Maintenance.

12. What is Re-engineering?

Improving existing software to enhance maintainability.

13. Reverse Engineering

Analyzing existing software to understand its structure.

14. Forward Engineering

Developing software from design to implementation.

PART B — 2 Marks Answers

1. State any two objectives of Software Quality Assurance

The objectives of **Software Quality Assurance (SQA)** are:

1. **Ensure software quality** by following defined standards and procedures during development.
2. **Detect defects early** in the development process to reduce errors and development cost.

Other objectives include improving development processes and ensuring customer satisfaction.

2. Differentiate between Verification and Validation

Verification

Verification checks whether the software is built according to specifications.

It answers the question “**Are we building the product right?**”

Done during development using reviews and inspections.

Validation

Validation checks whether the software meets user requirements.

It answers the question “**Are we building the right product?**”

Done after development using testing methods.

3. Write the purpose of Formal Technical Review (FTR)

The main purpose of **Formal Technical Review (FTR)** is to examine software artifacts such as design documents, code, and requirements to identify defects early.

Objectives of FTR include:

- Detect errors in logic, design, and implementation.
 - Ensure the software meets requirements.
 - Improve software quality before the next development phase.
-

4. List the characteristics of Software Reliability

The main characteristics of **software reliability** include:

- **Failure-free operation** for a specific period of time.
- **Consistent performance** under specified conditions.
- **Dependability and stability** during system operation.
- **Error tolerance and recovery capability.**

Reliable software reduces system failures and increases user confidence.

5. Write two benefits of ISO 9000 standards

Two major benefits of **ISO 9000 standards** are:

1. **Improves product quality** by ensuring standardized development processes.
2. **Increases customer confidence** by maintaining consistent quality management practices.

ISO certification also helps organizations gain international recognition.

6. What are the five maturity levels of CMM?

The **Capability Maturity Model (CMM)** defines five levels of process maturity:

1. **Initial Level** – Processes are unpredictable and poorly controlled.
 2. **Repeatable Level** – Basic project management practices are established.
 3. **Defined Level** – Standardized development processes are documented.
 4. **Managed Level** – Processes are measured and controlled using metrics.
 5. **Optimizing Level** – Continuous improvement of processes is practiced.
-

7. Write two principles of Six Sigma in software engineering

Two important principles of **Six Sigma** are:

1. **Reducing defects** in software processes to improve product quality.
2. **Improving process performance** through measurement and analysis.

Six Sigma focuses on achieving near-perfect quality using the **DMAIC methodology**.

8. List the components of an SQA Plan

The **Software Quality Assurance (SQA) Plan** includes the following components:

- Quality objectives
- Quality standards and procedures
- Roles and responsibilities
- Review and audit procedures
- Testing strategy
- Documentation standards
- Risk management

These components ensure that quality is maintained throughout the software development lifecycle.

9. Differentiate between Corrective and Adaptive Maintenance

Corrective Maintenance

Fixes errors or bugs found after software deployment.

Focuses on error correction.

Example: Fixing login errors.

Adaptive Maintenance

Modifies software to adapt to new environments or technologies.

Focuses on system compatibility.

Example: Updating software for a new operating system.

10. Differentiate between Reverse Engineering and Re-engineering

Reverse Engineering

Process of analyzing existing software to understand its design and structure.

Does not modify the system functionality.

Used when documentation is missing.

Re-engineering

Process of improving existing software to enhance performance or maintainability.

Modifies and improves the system structure.

Used to modernize or upgrade legacy systems.

PART C — 3 Marks Answers

1. Explain the concept of Software Quality

Software quality refers to the degree to which a software product meets specified requirements and satisfies the needs of users. High-quality software should perform its intended functions correctly, efficiently, and reliably.

Software quality is measured using several quality attributes such as:

- **Correctness** – Software produces accurate results according to requirements.
- **Reliability** – Software performs consistently without failure.
- **Efficiency** – Software uses system resources such as CPU and memory effectively.
- **Usability** – Software is easy to learn and use.
- **Maintainability** – Software can be easily modified and updated.

Maintaining software quality is very important because poor quality software can cause system failures, financial losses, and user dissatisfaction.

2. Explain Software Quality Assurance Activities

Software Quality Assurance (SQA) is a systematic process used to ensure that software development follows defined quality standards and procedures.

The main SQA activities include:

1. Establishing Quality Standards

Organizations define quality guidelines and standards that must be followed during software development.

2. Software Reviews

Documents, designs, and code are reviewed to detect defects early.

3. Testing and Verification

Different types of testing are performed to ensure that software works correctly.

4. Process Monitoring

The development process is monitored to ensure it follows defined standards.

5. Defect Management

Errors are identified, documented, and corrected.

SQA helps improve product quality, reduce errors, and ensure customer satisfaction.

3. Describe the Process of Formal Technical Review (FTR)

A Formal Technical Review (FTR) is a structured review process conducted by a team of technical experts to identify defects in software products.

The FTR process includes the following steps:

1. Planning

The review leader selects the review team and schedules the review meeting.

2. Preparation

Reviewers study the software documents before the meeting.

3. Review Meeting

Team members discuss defects and suggest improvements.

4. Rework

The developer corrects the identified defects.

5. Follow-Up

The review leader ensures that corrections have been implemented.

FTR helps detect errors early in development and improves software quality.

4. Explain Software Reliability and its Importance

Software reliability is the probability that a software system operates without failure for a specific period under specified conditions.

In simple words, reliability means the software works correctly and consistently without crashing.

Characteristics of reliable software include:

- Failure-free operation
- Consistent performance
- Dependability
- Error tolerance

Importance of software reliability:

- Reduces system failures
- Increases user trust and satisfaction
- Improves system stability
- Reduces maintenance cost

Reliability can be improved through proper design, testing, and error handling mechanisms.

5. Explain ISO 9000 Standards in Software Engineering

ISO 9000 is a family of international standards developed by the **International Organization for Standardization (ISO)** to ensure quality management in organizations.

ISO 9000 focuses on improving development processes and maintaining product quality.

Key features of ISO 9000:

- Standardized development processes
- Proper documentation of procedures
- Continuous quality improvement
- Focus on customer satisfaction

Benefits of ISO 9000 include:

- Improved product quality
- Better project management
- Increased customer confidence
- International recognition

Organizations that follow ISO standards can produce reliable and high-quality software products.

6. Explain the Capability Maturity Model (CMM)

The Capability Maturity Model (CMM) is a framework developed by the **Software Engineering Institute (SEI)** to improve software development processes.

CMM defines **five maturity levels**:

Level 1 – Initial

Processes are unpredictable and poorly controlled.

Level 2 – Repeatable

Basic project management practices are established.

Level 3 – Defined

Development processes are standardized and documented.

Level 4 – Managed

Processes are measured and controlled using metrics.

Level 5 – Optimizing

Continuous improvement of processes takes place.

CMM helps organizations improve productivity, quality, and process efficiency.

7. Explain Six Sigma Methodology

Six Sigma is a quality improvement methodology that focuses on reducing defects and improving process performance.

The goal of Six Sigma is to achieve very low defect rates.

Six Sigma follows the **DMAIC methodology**:

1. Define

Identify the problem and project objectives.

2. Measure

Measure the current performance of the process.

3. Analyze

Identify the root causes of defects.

4. Improve

Implement solutions to eliminate defects.

5. Control

Monitor the process to maintain improvements.

Six Sigma helps improve software quality, reduce errors, and increase efficiency.

8. Explain Types of Software Maintenance

Software maintenance refers to modifying software after it has been delivered to users.

There are **four types of software maintenance**:

1. Corrective Maintenance

Fixing errors or bugs found in software after deployment.

Example: Fixing login errors in an application.

2. Adaptive Maintenance

Modifying software to work with new environments such as new operating systems or hardware.

3. Perfective Maintenance

Improving system performance or adding new features.

4. Preventive Maintenance

Improving software structure to prevent future problems.

Software maintenance ensures that the system continues to meet user requirements.

9. Explain Reverse Engineering with Example

Reverse engineering is the process of analyzing an existing software system to understand its design and structure without changing its functionality.

It is commonly used when the documentation of a system is missing.

Example:

Analyzing a compiled application to understand its architecture and data flow.

Benefits:

- Helps understand legacy systems
 - Improves system documentation
 - Helps in system maintenance
-

10. Explain Re-Engineering and Forward Engineering

Software Re-Engineering

Re-engineering is the process of analyzing and modifying existing software to improve its quality, performance, or maintainability.

Activities include:

- Code restructuring
- Updating documentation
- Improving system design

Example: Improving an old banking software system.

Forward Engineering

Forward engineering is the process of developing software from high-level design to implementation.

Steps include:

1. Requirement analysis
2. System design
3. Coding
4. Testing
5. Deployment

Forward engineering represents the normal software development process.

PART D — 4 Marks Answers

1. Explain Software Quality Assurance and SQA Plan

Software Quality Assurance (SQA) is a systematic approach used to ensure that software development processes and products meet defined quality standards.

SQA focuses on **preventing defects rather than detecting them later**.

Objectives of SQA:

- Ensure compliance with quality standards
 - Improve development processes
 - Detect defects early
 - Deliver reliable software
-

Components of an SQA Plan

An SQA Plan describes how quality assurance activities will be carried out in a project.

Important components include:

- Quality objectives
- Quality standards
- Roles and responsibilities
- Review and audit procedures
- Testing strategy
- Documentation standards
- Risk management

The SQA plan acts as a roadmap to maintain software quality throughout the development lifecycle.

2. Software Reviews and Formal Technical Review

A software review is a process in which software artifacts such as requirements, design documents, and source code are examined to identify defects.

Reviews help improve software quality and reduce development cost.

Types of Reviews

1. Informal Review

Casual evaluation of documents or code.

2. Formal Technical Review (FTR)

A structured review conducted by technical experts.

Advantages of Reviews

- Early defect detection
 - Improved product quality
 - Better team communication
 - Reduced development cost
-

Disadvantages

- Requires time and effort
 - Needs experienced reviewers
-

3. Software Reliability and Quality Standards

Software reliability is the probability that software will operate without failure for a specific period under specific conditions.

Reliable software improves system stability and user trust.

Quality Standards

ISO 9000

International quality management standard focusing on process improvement.

Capability Maturity Model (CMM)

Five levels:

- 1 Initial
- 2 Repeatable
- 3 Defined
- 4 Managed
- 5 Optimizing

Six Sigma

Quality improvement method using DMAIC:

Define → Measure → Analyze → Improve → Control

These standards help organizations develop high-quality software systems.

4. Types of Software Maintenance

Software maintenance refers to modifying software after it is delivered.

Types include:

Corrective Maintenance

Fixing bugs or errors.

Adaptive Maintenance

Updating software for new environments.

Perfective Maintenance

Improving performance or adding features.

Preventive Maintenance

Improving software structure to avoid future problems.

Maintenance is important because software must evolve with changing technology and user requirements.

5. Re-Engineering, Reverse Engineering and Forward Engineering

Engineering Type	Description
Reverse Engineering	Understanding existing software structure
Re-Engineering	Improving existing software
Forward Engineering	Developing software from design to implementation

Reverse engineering helps understand legacy systems, while re-engineering improves them. Forward engineering is the normal software development process.